Voting Guide

The Zally shielded voting protocol, as deployed for Zcash coinholder polling

m@rek.onl

Abstract

This volume of the series — *Math Guide*, *Crypto Guide*, *Halo 2 Guide*, *Orchard Guide*, *Wallet Guide*, *Voting Guide*, *Tachyon Guide* — documents the shielded voting protocol deployed for Zcash coinholder polling. No specification of the protocol has merged anywhere; the commit-pinned implementation is the de facto specification, and this volume documents that implementation: ballot structure and eligibility proving against a snapshot of the note-commitment tree, the layered nullifier defences against double voting, threshold ElGamal tallying under a validator DKG, and — stated without euphemism — the trust assumptions and the one confirmed gap between the published description and the deployed circuit.

Contents

1 The protocol and its provenance

This volume documents the *Shielded Voting Protocol* — internally "Zally", shipped to users as *Coinholder Polling* — the first governance mechanism to run over Zcash's shielded pool in production. The protocol lets a holder of Orchard notes vote with the weight of their unspent funds, privately: the funds never move, the notes and addresses are never revealed, and only aggregate totals become public. The internal name appears in the client library's own description of itself ("the Zally governance protocol", `valargroup/zcash_voting` @ `624cf198`, `README.md`); the draft ZIP carries the public name (`draft-valargroup-shielded-voting.md`, zcash/zips PR #1200); the wallet changelog carries the product name (`zashi-android` @ `b4edd6f6`, `CHANGELOG.md`). This section fixes what shipped, where the design came from, and — decisive for every citation in this volume — what the specification actually is.

1.1 What shipped

In outline: a voting round pins a snapshot of the Zcash chain — an Orchard note-commitment root and a nullifier-set root. A holder proves in zero knowledge that they control notes which existed and were unspent at the snapshot, and delegates their quantised weight — one ballot per 0.125 ZEC (12,500,000 zatoshi per ballot, `voting-circuits/src/params.rs`) — to a round-scoped governance hotkey. The hotkey casts each vote as ElGamal-encrypted split shares on a dedicated permissioned chain, `svoted`, whose ballot surface is five messages (`vote-sdk/proto/svote/v1/tx.proto`); relayers reveal shares with timing decorrelation; and a per-round threshold *Election Authority* — a distributed key generation among the chain's validators — decrypts aggregates only (`vote-sdk/x/vote/keeper/keeper_tally.go`). The remainder of this volume constructs each of these objects in turn; here we record only their provenance.

Deployment. Coinholder Polling shipped in Zodl — the wallet formerly Zashi; the team left ECC in January 2026 and the application package identity is unchanged — version 3.5.0 on Android and iOS, released 2026-05-28, with signing support for the Keystone hardware wallet (`zashi-android` @ `b4edd6f6`, `CHANGELOG.md`, entry `[3.5.0 (1736)]`); a follow-up release, 3.5.2, followed on 2026-06-04 (ibid., entry `[3.5.2 (1742)]`). The first production use was the NU7 sentiment coinholder poll, open 2026-06-10 through 2026-06-24 with results published by 2026-06-29, run in parallel with a ZCAP poll of the same question (statuses checked 2026-07-08).

Authorship. Valar Group designed and implemented the protocol, led by Dev Ojha (GitHub `ValarDragon`); the draft ZIP is authored by GitHub user `p0mvn`, also of Valar Group. The upstream `orchard` crate gained the protocol's one in-tree dependency — the `unstable-voting-circuits` feature, which widens circuit internals (the NoteCommit, nullifier-derivation, and $\mathsf{Commit}^{\mathsf{ivk}}$ chips) for out-of-tree reuse — in a commit by Adam Tucker (zcash/orchard @ `c0b6b917`, 2026-04-23). ZODL engineers str4d (Jack Grigg), Daira-Emma Hopwood, and Kris Nuttycombe reviewed — they did not design — the architecture in Q1 2026 (ZODL retroactive grant application, ZcashCoinholderGrantsProgram issue #33); ZODL also built the wallet integration.

1.2 Lineage

Coin-weighted shielded polling on Zcash has one prior implementation: hhanh00's `zcash-vote`, a standalone application that ran the 2024 developer-fund poll and the April–May 2025 lockbox poll (`hhanh00/zcash-vote` @ `f05f5f25`, `src/election.rs`; it depends on a forked `orchard` at rev `75448e67` with a `vote` feature). Its ZIP — zcash/zips PR #853, "Coin Voting ZIP" — has been open since 2024-05-28 and remains unmerged as of 2026-07-08. Zally is independent of it:

neither codebase references the other (verified by search over both trees, 2026-07-08), and Zally reuses the upstream `orchard` crate through a feature flag where `zcash-vote` patches in its fork. The two share the core idea — pin a snapshot, prove note eligibility against it, and prevent double voting with a per-election nullifier — but Zally adds hotkey delegation through a dedicated note type (the Vote Authority Note), split ElGamal shares, a threshold Election Authority, relayers, private eligibility queries by single-server PIR, a dedicated chain, and hardware signing, and it lives inside the production wallet rather than beside it.

Two adjacent mechanisms are not ancestors. ZCAP, the Zcash Community Advisory Panel, polls an identified panel through Helios with no coin weighting; the June 2026 NU7 question ran through both instruments in parallel, which makes the distinction operational, not academic. Hopwood's draft `draft-ecc-onchain-accountable-voting.md` (created 2025-02-21, in the merged zips tree at `c6b70358`) targets the complement — accountable on-chain voting by identified keyholders — and shares no design lineage with either.

1.3 The source situation: code as the de facto specification

No normative document exists. The would-be specification, zcash/zips PR #1200 ("[ZIP TBD] Shielded Voting Protocol", opened 2026-03-05), is open and unmerged as of 2026-07-08, not updated since 2026-06-02, and carries a review comment flagging a section as "Underspecified"; the companion PIR draft, PR #1198, is likewise open. The design book the implementation itself cites — `valargroup/shielded-vote-book`, linked from the `zcash_voting` README — is private, as is its mirror `z-cale/shielded-vote-book` (both return 404 to anonymous access, verified 2026-07-08). The public documentation (`valargroup.gitbook.io/shielded-vote-docs`, fetched 2026-07-08) is prose: its circuit pages list public and private inputs and some Poseidon preimages, but omit the governance-nullifier, VAN-nullifier, and share-nullifier formulas, the nullifier-domain derivation, the proposal-bitmask width, and the indexed-Merkle-tree leaf model — and it misstates the ballot-quantisation rule, the sole open problem this volume records (the closing paragraph of this section).

The consequence governs this volume: the commit-pinned implementation of Table 1 is the de facto specification, and what we document is that implementation. Every governance-specific formula in the chapters that follow is read from these sources at these commits, never from designer prose; where the prose and the code disagree, the code is the protocol, and we flag the disagreement.

Repository	Commit	Date	Role
`valargroup/voting-circuits`	`4c39abd5`	2026-06-03	circuits: prover *and* verifier
`valargroup/vote-sdk`	`cb915f51`	2026-06-05	vote chain: validation, DKG, tally
`valargroup/zcash_voting`	`624cf198`	2026-06-07	client: proofs, scheduling, persistence
`valargroup/vote-nullifier-pir`	`603e4e3b`	2026-06-03	PIR: nullifier non-membership
`valargroup/spendability-pir`	`a924008d`	2026-06-04	PIR: spendability and witness queries
`zcash/orchard`	`c0b6b917`	2026-04-23	`unstable-voting-circuits` feature

Table 1: The de facto specification: the sources this volume documents, pinned by commit. Wallet-side integration is pinned separately: `zcash-android-wallet-sdk` @ `f386369e` depends on `zcash_voting =0.11.0` and `orchard =0.14.0` with the `unstable-voting-circuits` feature (`backend-lib/Cargo.toml`); `zashi-android` @ `b4edd6f6`, `zashi-ios` @ `8ab0e6ed`.

The single-crate caveat. Crate `voting-circuits` exposes both the provers and the verifiers of all three circuits (`verify_delegation_proof`, `verify_vote_proof`, and `verify_share_reveal_proof`, each in its circuit directory's `prove.rs`), and the chain consumes the same crate for validation: `vote-sdk/circuits/Cargo.toml` pins `voting-circuits` at version `0.8.0`, and `vote-sdk/x/vote/ante/validate.go` invokes it. Wallet and chain therefore

cannot drift apart — but their agreement is a tautology, not evidence of correctness. No independent second implementation exists that could catch a mis-transcribed relation, and no merged specification exists against which either side could be audited. The reader should weigh every constraint we state in later sections with this in mind.

Classification. Design-stage material in this volume obeys a three-bin discipline. Every claim is exactly one of: *specified* — a merged normative artifact exists and we cite it; *designed but unspecified* — the design is concrete and open-source but no normative artifact pins it, so we cite the implementation and state what a specification would need to fix; or an *open problem* — the design itself leaves the question unresolved. The bin is always visible in the prose.

Specified. Only the inherited primitives: NoteCommit, nullifier derivation, $\mathsf{Commit}^{\mathsf{ivk}}$, the Sinsemilla Merkle instance, RedPallas, the ZIP-244 sighash, and ZIP-32 key derivation — each specified in the Zcash protocol specification and the ZIPs, constructed in the *Orchard Guide* (§"Representation of a note: notes and note commitments", §"Prevention of double-spending: nullifiers", §"Keys: capabilities for spending and for viewing", §"Proof of ownership of *some* valid note: the commitment tree", §"Authorisation of the spend: RedPallas"), and reused unchanged through the `unstable-voting-circuits` feature (zcash/orchard @ `c0b6b917`). No governance-specific object has a specification of that rigor.

Designed but unspecified. Everything else this volume presents: the five-message ballot structure, the delegation relation with its indexed-Merkle-tree non-membership argument, the coordinator-posted snapshot roots, the Vote Authority Note and its synthetic-action authorisation, all four double-vote layers, the split-share ElGamal encoding and vote commitment, the relayer protocol and the trust it grants, the per-round DKG and chain-verified tally, and the PIR subsystem. For each we cite the pinned commit and file, and we note what PR #1200 would need to pin down for the object to change bins.

Open problems. One, and only one, was found: the ballot-quantisation relation enforced by the delegation circuit is strictly weaker than the exact floor division the public documentation states — the relation admits a one-ballot *under*-claim for roughly a third of note-value totals, while an over-claim remains impossible (`voting-circuits/src/delegation/circuit.rs`, `src/delegation/README.md` §8). We state and analyse the gap where delegation weight is constructed. It is the only spec-versus-code disagreement this volume records.

2 Ballots and eligibility

A voting round is a conversation in five messages on the vote chain, and admission to it is a zero-knowledge proof against a pinned Zcash snapshot. This section constructs both sides of that admission: the wire surface of a round (§2.1); the eligibility relation the delegation circuit proves — ownership of Orchard notes that existed at the snapshot and were unspent at it (§2.2, §2.3); the provenance of the two snapshot roots those proofs are checked against (§2.4); and the quantization that turns note value into voting weight (§2.5). Except where marked otherwise, every claim below sits in the *designed but unspecified* bin of §1.3: we read it from the commit-pinned sources of Table 1, and each subsection closes by recording its bin. The quantization carries this volume's single open problem; we state and prove it where it arises.

2.1 The five-message round

The ballot surface of the vote chain is five protobuf messages (`vote-sdk/proto/svote/v1/tx.proto`). In lifecycle order: a coordinator opens the round, holders delegate weight, governance hotkeys cast votes, helper servers reveal shares, and the round closes with a tally. The

three voter-facing messages (2–4) are not ordinary Cosmos transactions: they bypass the Cosmos transaction envelope entirely, and the chain authenticates each by its zero-knowledge proof — messages 2 and 3 additionally by a RedPallas signature; message 4 carries none — rather than by an account signature (`tx.proto:10-13`; `vote-sdk/x/vote/ante/validate.go`).

Message 1: `MsgCreateVotingSession`. The round-opening message carries the snapshot pin and the agenda: `snapshot_height`, `snapshot_blockhash`, `proposals_hash`, `vote_end_time`, the two ledger anchors `nullifier_imt_root` and `nc_root`, the proposal list, and display metadata (`tx.proto:36-48`). It is not a public RPC: it is absent from the `Msg` service and executes only as the payload of a threshold-approved coordinator action (`vote-sdk/x/vote/keeper/msg_server_coordinator_actions.go:294-298`). The handler derives the round identifier on-chain as one Poseidon hash over the creation height, the snapshot block hash and the proposals hash (each split into two field elements), the end time, and both anchors, so `vote_round_id` commits to the snapshot (`tx.proto:32-35`; `msg_server.go:30-48`).

Message 2: `MsgDelegateVote` (ZKP 1). The delegation message carries `rk` (the randomized spend-validating key), `spend_auth_sig` (RedPallas), `signed_note_nullifier` (the nullifier of the synthetic keystone note), `cmx_new` (the output note commitment), `van_cmx` (the Vote Authority Note commitment), `gov_nullifiers` (up to five governance nullifiers), `proof`, `vote_round_id`, and `sighash` — a *client-provided* 32-byte value over which the chain verifies the RedPallas signature without recomputing it (`tx.proto:55-65`). This section proves the eligibility half of the message; the Vote Authority Note is constructed in §3.1, and the synthetic keystone note with its signature binding — the ρ-chain through which the RedPallas signature commits to the whole delegation — in §3.2.

Message 3: `MsgCastVote` (ZKP 2). A cast vote carries `van_nullifier`, `vote_authority_note_new` (the successor Vote Authority Note), `vote_commitment`, `proposal_id`, `proof`, `vote_round_id`, the anchor height `vote_comm_tree_anchor_height`, `vote_auth_sig`, and `r_vpk`, a compressed Pallas point that the chain decompresses for the verifier; unlike message 2, the sighash here is recomputed on-chain (`tx.proto:72-84`, including the removed-field comments).

Message 4: `MsgRevealShare` (ZKP 3). A share reveal carries `share_nullifier`, `enc_share` — 64 bytes, one lifted-ElGamal ciphertext $C_1 \,\|\, C_2$ as compressed Pallas points — `proposal_id`, `vote_decision`, `proof`, `vote_round_id`, and `vote_comm_tree_anchor_height` (`tx.proto:89-97`). Helper servers, not wallets, submit it: the share nullifier is constructed in §3.3, and the helper protocol and the trust it grants are treated in §4.1.

Message 5: `MsgSubmitTally`. The closing message carries `vote_round_id`, `creator` — which the chain checks against the *block proposer*: the proposer's node injects the message during tallying, and the proto comment naming the session creator is unenforced (`x/vote/keeper/msg_server_tally_decrypt.go:77-79`; `app/prepare_proposal.go:194-212`) — and one `TallyEntry` {`proposal_id`, `vote_decision`, `total_value`, `decryption_proof`} per (proposal, decision) pair (`tx.proto:104-116`). The `total_value` field is denominated in ballots despite its proto comment saying zatoshi; we return to this in §3.6.

Classification. Designed but unspecified: the message set, field semantics, and transport exist only in `vote-sdk/proto/svote/v1/tx.proto` at commit `cb915f51` (Table 1). A specification would need to pin the byte-level encoding and length of every field, the validation order, the envelope-bypass transport, and the round-identifier derivation — the last currently fixed only by a proto comment and the FFI implementation it describes.

2.2 Eligibility: the note-bundle relation (ZKP 1)

To delegate weight, a holder must prove three things about a set of Orchard notes without revealing any of them: the notes are theirs; each existed at the snapshot; each was unspent at the snapshot. The delegation circuit proves the conjunction — one Halo 2/IPA circuit over Pallas at $K = 14$ (16,384 rows) with 14 public inputs (`voting-circuits/src/delegation/circuit.rs:111` and the module README, both at commit `4c39abd5`).

Bundle shape. The circuit carries exactly five note slots (`MAX_REAL_NOTES`); a wallet with fewer notes pads the remaining slots with zero-value notes, and one with more produces multiple delegation proofs. The fixed width is deliberate: every delegation publishes the same shape — five governance nullifiers at public-input offsets 8–12 — so the published proof does not reveal how many real notes it bundles (`src/delegation/README.md`, Inputs). The two ledger anchors enter as public inputs `nc_root` (offset 6) and `nf_imt_root` (offset 7); the nullifier domain dom (offset 13) is exposed for API compatibility but constrained in-circuit to $\mathsf{Poseidon}(\mathsf{tag}, \texttt{vote\_round\_id})$, where tag is the ASCII string `governance authorization` zero-padded to a Pallas base-field element (`src/domain_tags.rs`; `src/delegation/imt.rs:26-35`). Poseidon throughout the protocol is the P128Pow5T3 instance, width 3, rate 2 (`src/protocol_hash.rs`).

Per-slot conditions. Each slot proves six conditions, numbered 9–14 in the source[1] (`src/delegation/circuit.rs:1497-1805`):

- *Note commitment integrity* (condition 9). The prover witnesses the note plaintext $(\mathsf{g_d}, \mathsf{pk_d}, v, \rho, \psi)$, the trapdoor rcm, and the claimed commitment cm; the circuit recomputes $\mathsf{NoteCommit}_{\mathsf{rcm}}$ over the witnessed fields, constrains it equal to cm, and extracts $\mathsf{cmx} = \mathsf{Extract}(\mathsf{cm})$ as an internal wire. The commitment scheme is the deployed Orchard NoteCommit — specified bin, reused unchanged through the `unstable-voting-circuits` feature (§1.3).

- *Address ownership* (condition 11). The circuit checks $\mathsf{pk_d} = [\mathsf{ivk}^*]\,\mathsf{g_d}$, where ivk^* is selected by a witnessed boolean is_internal through the custom gate `q_scope_select`, $\mathsf{ivk}^* = \mathsf{ivk} + \mathsf{is\_internal} \cdot (\mathsf{ivk}_{\mathsf{int}} - \mathsf{ivk})$; both candidates are derived in-circuit by $\mathsf{Commit}^{\mathsf{ivk}}$ from the witnessed $\mathsf{r}_{\mathsf{ivk}}$ and its internal-scope counterpart (condition 5, global). Change notes therefore carry weight on equal terms with externally received ones. A wrong scope flag selects the wrong key and the ownership check rejects.

- *Snapshot membership* (condition 10). The circuit computes the Sinsemilla Merkle root from the leaf cmx over the witnessed 32-level authentication path, and the per-note gate `q_per_note` enforces
$$v \cdot (\mathsf{root} - \mathsf{anchor}) = 0,$$
with anchor the public `nc_root` (`circuit.rs:1777-1798`). A real note ($v \neq 0$) must resolve to the anchor; a zero-value padding note — which has no leaf in the Zcash tree — satisfies the gate vacuously.

- *Real nullifier* (condition 12). The circuit derives the note's true Orchard nullifier $\mathsf{nf} = \mathsf{DeriveNullifier}_{\mathsf{nk}}(\rho, \psi, \mathsf{cm})$ — the deployed Orchard rule, constructed in the *Orchard Guide* (§"Prevention of double-spending: nullifiers") — in-circuit and *never publishes it*: it exists only as a private wire feeding conditions 13 and 14 (`circuit.rs:1682-1701`).

[1] The headline counts disagree — `circuit.rs` line 3 says "all 14 conditions" but line 108 "all 15 conditions"; `README.md` lines 3 and 52 say "15" and "conditions 9–15" — yet both files enumerate exactly conditions 1–14 and neither defines a condition 15: the README's numbered sections run 1 through 14 and the per-slot list is 9–14 throughout. We state the total as fourteen — eight keystone-global conditions plus six per-slot conditions instantiated five times — and record the "15" as a stale count in the sources.

- *Snapshot non-spentness* (condition 13). The circuit proves non-membership of nf in the snapshot's nullifier set under the public `nf_imt_root` — the construction of §2.3. The root equality is unconditional: padding notes, too, must carry a valid non-membership proof (`README.md` §13).
- *Governance nullifier* (condition 14). The circuit computes and publishes $\mathsf{nf}^{\mathsf{gov}} = \mathsf{Poseidon}(\mathsf{nk}, \mathsf{dom}, \mathsf{nf})$ at the slot's public-input offset. Keyed by the private nk, the published value stays unlinkable to nf even after the note is later spent on the main chain (`circuit.rs:1703-1729`). Its role in double-vote prevention is treated in §3.3. Chain-side, the delegation handler applies the same atomic check-and-set to all five published slots, padding included (`vote-sdk/x/vote/keeper/msg_server.go:148-154`); a padding slot's nullifier enters the same round-scoped set, unique because the wallet samples fresh ρ and `rseed` per synthetic padding note (`src/delegation/builder.rs`, `build_padding_slot`), and a re-used padding witness is rejected as a duplicate delegation — a completeness cost, never a soundness one.

The conjunction across a slot reads: the prover's key material owns a note whose commitment lies in the Zcash note-commitment tree at the snapshot (membership under `nc_root`) and whose nullifier had not been revealed by the snapshot (non-membership under `nf_imt_root`) — existed, and unspent, at the pinned height.

Classification. The relation as a whole is designed but unspecified: it exists only as the circuit at `voting-circuits` @ `4c39abd5`, which is simultaneously the prover and the verifier (§1.3). Its ingredients $\mathsf{NoteCommit}$, nullifier derivation, $\mathsf{Commit}^{\mathsf{ivk}}$, and the $\mathsf{Sinsemilla}$ Merkle instance are the specified bin, inherited through zcash/orchard @ `c0b6b917`. A specification would need to pin the public-input layout, the custom-gate semantics, the fixed five-slot shape, and the padding-note rules.

2.3 Snapshot non-spentness: the Poseidon indexed Merkle tree

Membership alone would let a spent note vote: the note-commitment tree is append-only and a spent note's commitment remains in it forever. Snapshot eligibility therefore needs a second, negative statement — the note's nullifier is *not* in the set of nullifiers revealed up to the snapshot height. Proving non-membership in a Merkle set requires structure beyond a plain accumulator; the protocol uses an indexed Merkle tree over the sorted nullifier set.

Construction. The tree is Poseidon-based with depth 29 (`IMT_DEPTH`, `voting-circuits/src/delegation/imt.rs:19`). Each leaf is a *punctured range* — a triple of sorted boundaries, hashed and covering the punctured open interval

$$\mathsf{leaf} = \mathsf{Poseidon}(\mathsf{nf}_{\mathrm{lo}}, \mathsf{nf}_{\mathrm{mid}}, \mathsf{nf}_{\mathrm{hi}}), \qquad (\mathsf{nf}_{\mathrm{lo}}, \mathsf{nf}_{\mathrm{hi}}) \setminus \{\mathsf{nf}_{\mathrm{mid}}\},$$

so the set members appear only as interval boundaries or interior punctures, and everything strictly between them is certified absent. In-circuit, non-membership of the private nf costs 31 Poseidon calls (2 for the leaf, 29 for the path) and proves (`imt.rs:51-70`; `src/delegation/README.md` §13):

1. the leaf authenticates: the 29-level path from the leaf hash resolves to the public `nf_imt_root`;
2. the nullifier lies strictly inside the interval: the offsets $x_{\mathrm{lo}} = \mathsf{nf} - \mathsf{nf}_{\mathrm{lo}} - 1$ and $x_{\mathrm{hi}} = \mathsf{nf}_{\mathrm{hi}} - \mathsf{nf} - 1$ are each range-checked to $[0, 2^{250})$; and
3. the nullifier misses the puncture: $(\mathsf{nf} - \mathsf{nf}_{\mathrm{mid}}) \cdot \delta = 1$ for a witnessed inverse δ.

What it excludes — and what it does not. The statement excludes exactly the nullifiers revealed on the Zcash chain up to the snapshot height: the note was unspent when the snapshot was taken. It excludes nothing after that height — a note spent the block after the snapshot still delegates its weight, because snapshot voting fixes the electorate at a single height by design. Two soundness assumptions live *outside* the circuit (`imt.rs:56-70`; `README.md` §13). First, the tree contract: the circuit checks only the one authenticated leaf, so the committed leaves must form the canonical punctured-range tree for the set — sorted, deduplicated, non-overlapping except for shared boundaries. A malformed tree in which some set member lies strictly inside another leaf's interval would certify a spent note as unspent, and no per-leaf gate can detect the global overlap. Second, the span bound: the 250-bit offset checks require $\mathsf{nf}_{\mathrm{hi}} - \mathsf{nf}_{\mathrm{lo}} \leq 2^{250}$ in canonical field order, which the builder guarantees by pre-populating 33 sentinel nullifiers spaced at most 2^{249} apart, plus $p - 1$ to close the tail (the `SpacedLeafImtProvider` strategy). The tree is built by the PIR operator and its root posted by the coordinator — which is where the next subsection picks up.

Classification. Designed but unspecified (`voting-circuits` @ `4c39abd5`). A specification would need to pin the leaf model, the sentinel schedule, and above all the canonical-tree contract — including which party is accountable for the tree's well-formedness, since the circuit cannot check it.

2.4 The snapshot roots: posted, never verified

Both anchors originate off-chain, in coordinator tooling. The note-commitment root `nc_root` is recomputed by Rust FFI as the Sinsemilla hash of an Orchard frontier fetched from a public lightwalletd server; the nullifier root `nullifier_imt_root` is fetched from the running PIR service (`vote-sdk/api/snapshot.go:44-66`). The coordinator posts both in message 1; the handler stores them verbatim in the round record and folds them into `vote_round_id` (`x/vote/keeper/msg_server.go:36-138`); and at validation the ante handler looks the stored roots up from the round and passes them to the FFI verifier as the public inputs of ZKP 1 (`x/vote/ante/validate.go:147-177`).

Nothing on the vote chain verifies that the posted roots match real Zcash state: `svoted` runs no Zcash light client, and the circuit documentation is explicit that it "proves statements relative to the public inputs supplied by the verifier; it does not authenticate where those inputs came from" (`voting-circuits/src/delegation/README.md`, Public Input Provenance). The trust structure that results is asymmetric. Honest wallets are safe from a *wrong* snapshot in the sense that their proofs simply fail: their witnesses — Merkle paths and IMT leaves — come from the real chain and cannot resolve to fabricated roots. And the roots are public, so anyone with a Zcash node can audit them after the fact. But nothing prevents a coordinator quorum from pinning fabricated roots for which it alone holds matching witnesses, minting arbitrary voting weight that the chain would accept; detection is post hoc and protocol-external. The trust root for eligibility is therefore the coordinator threshold together with the PIR operator that builds the nullifier tree of §2.3.

Classification. Designed but unspecified, with an explicit trust assumption that must be flagged wherever eligibility is discussed. A specification would need either chain-side verification (a light client or a succinct proof that the roots match a Zcash block) or, failing that, a normative audit procedure with a defined challenge window.

2.5 Weight: ballot quantization

Delegated weight is denominated in ballots. The circuit sums the five slot values into $v_{\mathrm{total}} = \sum_{i=1}^{5} v_i$ (internal wires produced by condition 9, consumed by conditions 7 and 8;

`src/delegation/README.md`, Inputs), and one ballot represents $D = 12{,}500{,}000$ zatoshi (0.125 ZEC): the constant `BALLOT_DIVISOR` (`voting-circuits/src/params.rs:14`). The honest ballot count is $q = \lfloor v_{\text{total}}/D \rfloor$.

Condition 8 (`src/delegation/circuit.rs:1290-1427`) witnesses the count q (`num_ballots`) and a remainder r as *free advice*, and constrains

$$q \cdot D + r = v_{\text{total}}, \qquad 0 \le r < 2^{24}, \qquad 1 \le q \le 2^{30}. \tag{1}$$

The range checks run through a 10-bit lookup table: the circuit multiplies r by 2^6 and checks the product to 30 bits (so $r < 2^{24}$), and checks $q - 1$ to 30 bits directly — which enforces both bounds at once, since $q = 0$ wraps $q - 1$ to $p - 1 \approx 2^{254}$ and fails. The lower bound $q \ge 1$ doubles as a minimum stake: a bundle worth less than 0.125 ZEC admits no valid witness. The upper bound is safe slack: 2^{30} ballots is about 134 million ZEC, far above the 21 million supply. The remainder bound 2^{24} is the tightest power-of-two-aligned bound that keeps completeness — every honest $r < D$ fits — but it is *looser* than D, and that slack admits a second witness.

Proposition 2.1 (One-ballot under-claim window)**.** *Fix v_{total} and let $\hat{q} = \lfloor v_{\text{total}}/D \rfloor$ and $\hat{r} = v_{\text{total}} \bmod D$ form the honest witness, assumed valid ($\hat{q} \ge 1$). The pairs satisfying the constraints of Equation (1) are exactly $(\hat{q}, \hat{r})$ together with $(\hat{q} - 1,\ \hat{r} + D)$, the latter admissible if and only if*

$$\hat{r} < 2^{24} - D = 4{,}277{,}216 \qquad \text{and} \qquad \hat{q} \ge 2.$$

In particular, a prover can claim exactly one ballot fewer than floor division yields — for approximately 34.2% of totals under a uniform residue model, since $(2^{24} - D)/D \approx 0.3422$ — and can never claim more.

Proof. Suppose (q_1, r_1) and (q_2, r_2) both satisfy Equation (1) for the same v_{total}. Subtracting the two instances of the linear constraint gives $(q_1 - q_2)\, D \equiv r_2 - r_1 \pmod{p}$. The range checks bound the canonical representatives: $|q_1 - q_2| \cdot D \le (2^{30} - 1)D < 2^{54}$ and $|r_2 - r_1| < 2^{24}$, both far below $p > 2^{254}$, so the congruence holds over the integers. Then $|q_1 - q_2| = |r_2 - r_1|/D < 2^{24}/D < 2$, forcing $|q_1 - q_2| \in \{0, 1\}$. The case 0 gives $r_1 = r_2$. The case $q_1 = q_2 + 1$ gives $r_2 = r_1 + D$, which is admissible precisely when $r_2 < 2^{24}$ (the remainder range check) and $q_2 \ge 1$ (the count range check) — for the honest pair, the stated window. The over-claim direction from the honest witness, $(\hat{q} + 1, \hat{r} - D)$, requires $\hat{r} - D < 0$; its canonical field representative is at least $p - D$, which fails the shifted 30-bit range check. No further pairs exist. □

Impact and status. The deviation is self-harming: the only dishonest witness *under*-claims, discarding one ballot of the prover's own weight, and aggregate totals consequently err low, never high. No downstream check repairs or re-derives the count: the Vote Authority Note commitment (condition 7) hashes whatever q the prover chose, and the vote-proof circuit later opens the same value. The gap is documented by the implementers themselves — `src/delegation/README.md` §8, "Soundness scope", carries the analysis above together with two single-constraint tightenings (a 24-bit check on $(D - 1) - r$, or on $r + (2^{24} - D)$, either of which pins $r \in [0, D)$ at a cost of roughly 3–4 rows) — and it is unfixed at commit `4c39abd5`. The public documentation states exact floor division (gitbook ZKP 1 page, fetched 2026-07-08), which the circuit does not enforce: this is the single spec-versus-code disagreement recorded in this volume (§1.3), conservative in direction — the code is the protocol, and the code is weaker.

Classification. The quantization relation as implemented — Equation (1) — is designed but unspecified. The gap between it and the stated floor-division rule is this volume's one *open problem*: the design intends exact division, the circuit proves less, and the tightening exists on paper only. A specification must either adopt the weaker relation and its under-claim window explicitly, or mandate one of the documented tightening constraints.

3 Vote encryption, nullifiers, and tally

Delegation leaves the governance hotkey holding a Vote Authority Note (VAN): a blinded commitment to the hotkey address, a ballot count, and a per-proposal authority mask. Three problems remain. The hotkey must cast a vote whose weight never appears in public; the protocol must stop every form of double-voting — the same coins delegated twice, the same VAN spent twice, the same proposal voted twice along a chain of successor VANs, the same encrypted share counted twice; and the aggregate totals must become public in a form every chain node verifies, without any single party ever holding the decryption key. This section develops the answers in order: the VAN itself, the synthetic keystone note whose signature authorises its minting, a four-layer double-vote discipline, a sixteen-share lifted-ElGamal encoding, a per-round threshold Election Authority, and a chain-verified tally.

Classification. Every governance-specific object in this section is *designed but unspecified*: the normative artifact is the implementation, cited commit-pinned — `valargroup/voting-circuits` at `4c39abd5` (2026-06-03) and `valargroup/vote-sdk` at `cb915f51` (2026-06-05). File paths beginning `src/` refer to the former; paths beginning `x/vote/`, `crypto/`, or `proto/` to the latter. The would-be specification (zcash/zips PR #1200, opened 2026-03-05) remains unmerged as of 2026-07-08, and the single `voting-circuits` crate both proves and verifies, so no independent implementation cross-checks these formulas. The *specified* bin holds only the inherited Orchard primitives — $\mathsf{NoteCommit}$, nullifier derivation, $\mathsf{Commit}^{\mathsf{ivk}}$, RedPallas, the Sinsemilla Merkle instance — reused through the `orchard` crate's `unstable-voting-circuits` feature (zcash/orchard commit `c0b6b917`). One *open problem* reaches into this section from the delegation circuit; Remark 3.3 flags it where it lands.

Throughout this section, $\mathsf{Poseidon}$ denotes the P128Pow5T3 instance over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (width 3, rate 2), applied with the fixed-length padding of the stated arity. Domain separation uses two encodings, both pinned by unit tests (`src/domain_tags.rs`): small numeric tags (0 for VAN commitments, 1 for vote commitments), and short ASCII tags — `"governance authorization"`, `"vote authority spend"`, `"share spend"` — zero-padded to 32 bytes and read as little-endian integers, which are canonical in $\mathbb{F}_{p_{\mathsf{Pallas}}}$ because the top byte stays zero. We abbreviate the ASCII tags T_{gov}, T_{spend}, T_{share}, and write rid for the voting round identifier and pid for the proposal identifier.

3.1 The Vote Authority Note

The VAN is the vote chain's carrier of authority: one field element that commits to who may vote (the hotkey address), with how much weight (the ballot count), in which round, and on which proposals (the authority mask).

Definition 3.1 (VAN commitment)**.** Let $(\mathsf{g_d}, \mathsf{pk_d})$ be the hotkey's diversified address in the voting key hierarchy, n_b the ballot count, $\mathsf{auth} \in [0, 2^{16})$ the proposal-authority mask, and $\mathsf{r_{van}}$ a uniformly random blind. The VAN commitment is the two-layer hash

$$\mathsf{van} := \mathsf{Poseidon}\big(\mathsf{core},\ \mathsf{r_{van}}\big), \qquad \mathsf{core} := \mathsf{Poseidon}\big(0,\ x(\mathsf{g_d}),\ x(\mathsf{pk_d}),\ n_b,\ \mathsf{rid},\ \mathsf{auth}\big),$$

with $x(\cdot)$ the x-coordinate (`src/gadgets/van_integrity.rs`, lines 61–78; one gadget is shared by the delegation and vote-proof circuits).

The two layers divide labour. The inner layer is structural, and its preimage is low-entropy: the round id is public, the mask at mint is a fixed constant, and ballot counts are small integers — a bare inner hash would admit dictionary attacks on the weight and the address. The outer layer closes them with the uniform blind (module documentation, `src/gadgets/van_integrity.rs`, lines 23–27).

The address enters by x-coordinates alone, which identifies $(\mathsf{g_d}, \mathsf{pk_d})$ only up to coordinated negation. The construction is safe because delegation additionally binds the full points through NoteCommit into the public $\mathsf{cmx}_{\text{new}}$; the module documentation warns that any future caller dropping that side constraint must widen the preimage to full points (`src/gadgets/van_integrity.rs`, lines 14–22).

The mask auth is a 16-bit bitmask minted at $2^{16} - 1 = 65535$ — all bits set, a constant baked into the verification key (`src/delegation/circuit.rs`, lines 162 and 1464–1467). Bit 0 is a permanent sentinel that no vote may consume, so usable bits are 1 through 15 and a round supports at most 15 proposals (`src/vote_proof/circuit.rs`, lines 123–143).

The vote chain keeps one per-round Poseidon Merkle tree of depth 24 (`src/params.rs`, line 25; per-round state in `x/vote/keeper/keeper_voting.go`, lines 72–80): delegation appends the freshly minted VAN commitment; each cast appends the successor VAN and then the vote commitment. The reduced depth is a sizing choice — governance produces one leaf per delegation plus two per vote, well within $2^{24} \approx 1.7 \times 10^7$ leaves (`src/params.rs`, lines 16–25).

3.2 Keystone authorisation: the synthetic signed note

The delegation message is authorised by a RedPallas spend-auth signature under the randomized key rk (§2.1, message 2), and the intended signer is a Keystone-class hardware wallet, which signs only Orchard Actions. The voting client therefore wraps the delegation in an Orchard-Action shape around a *synthetic* keystone note: a 1-zatoshi note that exists on no chain — the circuit witnesses no Merkle path for it and checks no anchor; the value is 1 rather than 0 only because the wallet UI does not render zero-value spends (`src/delegation/README.md`, "Integration: the Keystone (signed) note is synthetic").

Conditions 1–6 of the delegation circuit govern this branch (`src/delegation/circuit.rs`, lines 16–29); the load-bearing pair is conditions 3 and 2. Condition 3 forces the keystone note's ρ to a Poseidon hash of everything the delegation publishes,

$$\rho_{\text{signed}} = \mathsf{Poseidon}\big(\mathsf{cmx}_1, \ldots, \mathsf{cmx}_5,\ \mathsf{van},\ \mathsf{rid}\big),$$

the arity-7 fixed-length instance (`rho_binding_hash`, `circuit.rs:176-190`). Condition 2 then derives the keystone nullifier by the deployed Orchard rule — constructed in the *Orchard Guide* (§"Prevention of double-spending: nullifiers") —

$$\mathsf{nf}_{\text{signed}} = \mathsf{DeriveNullifier}_{\mathsf{nk}}\big(\rho_{\text{signed}},\ \psi_{\text{signed}},\ \mathsf{cm}_{\text{signed}}\big),$$

and exposes it as a public input. The chain verifies the RedPallas signature under rk over the *client-provided* sighash and checks that the proof's $\mathsf{nf}_{\text{signed}}$ equals the wrapping Action's nullifier (`x/vote/ante/validate.go`), so the hardware wallet's signature commits — through the chain $\mathsf{rk} \to \mathsf{nf}_{\text{signed}} \to \rho_{\text{signed}} \to \mathsf{van}$ — to the five bundle commitments, the VAN, and the round identifier. Removing any link breaks the binding; this chain is why the chain-side acceptance of a client-provided sighash is safe. The remaining conditions are supporting: condition 4 proves $\mathsf{rk} = [\alpha]\,\mathsf{SpendAuthG} + \mathsf{ak}$ for a witnessed randomizer α (the *Orchard Guide*, §"Authorisation of the spend: RedPallas"); condition 5 ties ak, nk, and $\mathsf{r_{ivk}}$ to the same $\mathsf{Commit}^{\mathsf{ivk}}$ chain the real-note slots use (§2.2, condition 11); conditions 1 and 6 recompute the keystone and output note commitments from witnessed openings — knowledge checks, not membership checks (`src/delegation/README.md`, the three-bucket audit note).

3.3 Double-vote prevention: four layers

A ballot can be replayed at four distinct joints: the Zcash note (delegate it twice), the VAN (spend it twice), the proposal bit (vote the same proposal again through a successor VAN), and the encrypted share (feed it to the accumulator twice). One defence guards each joint. The first, second, and fourth are nullifiers; chain-side, each kind lands in its own type- and round-scoped set

under an atomic check-and-set (`x/vote/keeper/keeper_voting.go`, lines 19–48; type constants `x/vote/types/keys.go`, lines 96–101). The third is arithmetic, enforced in-circuit.

Layer 1: the governance nullifier (delegation). Each delegated note exposes

$$\mathsf{nf}^{\mathsf{gov}} := \mathsf{Poseidon}\big(\mathsf{nk},\ \mathsf{dom},\ \mathsf{nf}\big), \qquad \mathsf{dom} := \mathsf{Poseidon}\big(T_{\mathsf{gov}},\ \mathsf{rid}\big),$$

where nk and nf are the *Zcash* note's nullifier key and real Orchard nullifier, both computed in-circuit — the real nullifier is never published (`src/delegation/circuit.rs`, lines 1703–1729). The domain dom is a public input, but the circuit re-derives it from the public round id and constrains equality (`src/delegation/circuit.rs`, lines 1041–1055), so a prover cannot substitute a foreign domain to escape the round scoping. The chain rejects duplicates in the governance-typed, round-scoped set. The attack closed: delegating the same unspent-at-snapshot note into two bundles in one round, which would double its weight. Keying by the secret nk serves privacy: when the note is later spent on mainnet and its real nullifier becomes public, no observer can link the pair — the governance trail does not deanonymise the payment trail.

Layer 2: the VAN nullifier (casting). Spending a VAN publishes

$$\mathsf{nf}^{\mathsf{van}} := \mathsf{Poseidon}\big(\mathsf{nk}_v,\ T_{\mathsf{spend}},\ \mathsf{rid},\ \mathsf{van}_{\mathrm{old}}\big),$$

where nk_v is the voting hierarchy's nullifier-deriving key, proved to belong to the witnessed address through the $\mathsf{Commit}^{\mathsf{ivk}}$ chain (`src/vote_proof/circuit.rs`, lines 203–222). A cast proves membership of $\mathsf{van}_{\mathrm{old}}$ in the round tree without identifying the leaf, and this nullifier is the only spend mark. The attack closed: re-spending the old VAN, whose mask still carries the bit the vote just consumed — without the nullifier, one delegation would vote the same proposal arbitrarily often from the same leaf.

Layer 3: the authority-mask decrement (casting). Layer 2 kills the parent, not the lineage: every cast mints a successor VAN, and a successor still carrying the consumed bit would re-enable the same vote. Condition 6 of the vote-proof circuit therefore forces

$$\mathsf{auth}_{\mathrm{new}} = \mathsf{auth}_{\mathrm{old}} - 2^{\mathsf{pid}}.$$

Field arithmetic cannot express a variable exponent as a polynomial gate, so the prover witnesses 2^{pid} and a lookup over the table $\{(j, 2^j)\}_{j=0}^{15}$ proves it; a field-inverse gate excludes $\mathsf{pid} = 0$ (the sentinel); a 16-bit decomposition of $\mathsf{auth}_{\mathrm{old}}$ with a one-hot selector anchored at pid proves the consumed bit was actually set ($\sum_j \mathsf{sel}_j b_j = 1$), and the recomposition returns $\mathsf{auth}_{\mathrm{new}}$ with exactly that bit cleared, implicitly range-checked to 16 bits (`src/vote_proof/gadgets/authority_decrement.rs`, lines 34–93). The successor VAN commits $\mathsf{auth}_{\mathrm{new}}$ and re-enters the tree. The invariant bought by layers 2 and 3 together: one delegation yields at most one vote per proposal, and at most 15 proposals per round.

Layer 4: the share nullifier (reveal). Revealing the i-th encrypted share of a vote publishes

$$\mathsf{nf}_i^{\mathsf{share}} := \mathsf{Poseidon}\big(T_{\mathsf{share}},\ \mathsf{vc},\ i,\ \mathsf{blind}_i\big),$$

where vc is the vote commitment (§3.4) and blind_i the share-commitment blind (`src/share_reveal/circuit.rs`, lines 161–187). The attack closed: the tally accumulator is a raw homomorphic sum (§3.6), so re-submitting one accepted ciphertext would double-count its weight; the share-typed set admits each (vc, i) once. The blind is load-bearing for privacy rather than soundness: the vote commitment is public in the cast message and i ranges over sixteen values, so an unblinded nullifier could be evaluated by any observer against every commitment in the tree, linking each reveal to its vote by dictionary test — collapsing exactly the anonymity that the reveal proof's non-identifying membership provides. Blinds never appear on chain.

3.4 Vote encoding: sixteen lifted-ElGamal shares

Casting must convey weight to the tally without publishing it per vote. The instrument is additively homomorphic encryption to a key that no single party holds (§3.5).

Definition 3.2 (Vote share encryption)**.** Fix the generator

$$G := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard}, \texttt{G}),$$

Orchard's spend-authorisation base. A vote on proposal pid splits its ballot count n_b into sixteen prover-chosen shares $v_0, \ldots, v_{15}$ subject to

$$\sum_{i=0}^{15} v_i = n_b, \qquad v_i \in [0, 2^{30}),$$

and encrypts each under the round's Election Authority key $\mathsf{ea}_{\mathsf{pk}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$:

$$C_{1,i} := [r_i]\, G, \qquad C_{2,i} := [v_i]\, G + [r_i]\, \mathsf{ea}_{\mathsf{pk}},$$

with fresh nonzero randomness r_i (`src/gadgets/elgamal.rs`; sum and range are conditions 8 and 9, `src/vote_proof/circuit.rs`, lines 416–457 and 1026–1036).

The scheme is *lifted*: the message enters the exponent, so componentwise addition of ciphertexts encrypts the sum of plaintexts and the accumulator needs no key. The price is paid at decryption, which yields $[v]\, G$ and recovers v only as a bounded discrete logarithm — the 2^{28} bound of §3.6. Reusing $\mathsf{SpendAuthG}$ avoids introducing a second fixed base; the in-circuit 22-window short-scalar tables share the same generator (`src/gadgets/elgamal.rs`, lines 57–66). The range check closes wraparound: shares are field elements, and without it a decomposition could encode negative shares as large residues while still satisfying the sum constraint. Sixteen shares serve weight privacy under staggered reveal: the prover-chosen random decomposition leaves no weight fingerprint on any single revealed ciphertext (`src/vote_proof/circuit.rs`, lines 418–423). Client-side, the randomness r_i and the blinds derive from a BLAKE2b-512 PRF with personalisation `ZcashVote_Expand` under distinct domain bytes (`src/domain_tags.rs`, lines 28–41).

The cast binds everything into one public field element:

$$\mathsf{vc} := \mathsf{Poseidon}\big(1,\ \mathsf{rid},\ h_{\mathsf{sh}},\ \mathsf{pid},\ d\big), \qquad h_{\mathsf{sh}} := \mathsf{Poseidon}\big(\mathsf{sc}_0, \ldots, \mathsf{sc}_{15}\big),$$

$$\mathsf{sc}_i := \mathsf{Poseidon}\big(\mathsf{blind}_i,\ x(C_{1,i}),\ x(C_{2,i}),\ y(C_{1,i}),\ y(C_{2,i})\big),$$

where d is the vote decision (`src/gadgets/vote_commitment.rs`, lines 42–55; `src/shares_hash.rs`, lines 39–61). The y-coordinates are included deliberately: an x-coordinate identifies a point only up to sign, and hashing both coordinates blocks ciphertext sign-malleability. The leading tag 1 separates vote commitments from VANs (tag 0) in the shared tree and is baked into the verification key, so an honest verifier cannot be driven to confuse the two leaf kinds (`src/gadgets/vote_commitment.rs`, lines 11–13). A cast message reveals vc, the VAN nullifier, the successor VAN, and pid — neither the decision nor the weight. Note that the cast publicly links vc to $\mathsf{nf}^{\mathsf{van}}$, so per-vote weight privacy rests on the encryption, not on unlinkability.

Remark 3.3 (Quantization gap, propagated)**.** The sum condition ties $\sum_i v_i$ to the ballot count n_b minted at delegation, so its soundness inherits the delegation circuit's quantization relation: the constraint $n_b \cdot 12{,}500{,}000 + \mathsf{rem} = v_{\text{total}}$ with $\mathsf{rem} < 2^{24}$ is strictly weaker than floor division and admits a one-ballot *under*-claim for roughly 34% of note values; an over-claim is impossible (Proposition 2.1 in §2.5; `src/params.rs`, lines 9–13; `src/delegation/README.md`, §8). This is the protocol's one open problem — the public design prose states exact floor division, the code does not enforce it.

3.5 The Election Authority: a per-round Joint–Feldman DKG

The tally requires a key that opens aggregates; any party holding that key alone could equally open each accumulated share the moment it lands on chain. The protocol therefore splits the round key across the bonded validator set by a Joint–Feldman distributed key generation, and re-runs the ceremony every round, so compromise of one round's key reaches no other round and the validator set may change between rounds.

Each ceremony validator — registered through a whitelisted Pallas-key path — samples a secret polynomial of degree $t - 1$ over the Pallas scalar field, posts its t Feldman coefficient commitments (Pallas points) on chain, and delivers to every other validator its evaluation share, encrypted under ECIES: an ephemeral Pallas Diffie–Hellman, the symmetric key SHA-256($E \,\|\, S_x$) over the compressed ephemeral point and the shared-secret x-coordinate, and ChaCha20-Poly1305 with a zero nonce — safe because each envelope uses a fresh ephemeral key (`crypto/ecies/ecies.go`, lines 21–33). The chain validates point well-formedness and the commitment count (`x/vote/keeper/msg_server_ceremony.go`, lines 137–149). When the n-th contribution arrives, the chain sums the commitment vectors componentwise; the round key $\mathsf{ea}_{\mathsf{pk}}$ is the combined constant term, and each validator's verification key VK_i is the combined commitment polynomial evaluated at i in the exponent (`x/vote/keeper/msg_server_ceremony.go`, lines 224–269). No dealer ever holds the round secret: it exists only as the sum of shares.

The parameters are fixed in code: reconstruction threshold $t = \max(2, \lfloor (2n + 2)/3 \rfloor)$ for $n \geq 2$ validators (the degenerate $n = 1$, $t = 1$ case exists for local testing), ceremony finalization quorum $\lceil 4n/5 \rceil$ acknowledgements — stricter than t for most n — with non-ackers stripped from the ceremony and non-participants jailed (`x/vote/keeper/keeper_ceremony.go`, lines 46–79; `keeper_ceremony_jailing.go`).

The known key-bias attack on plain Joint–Feldman applies: contributions arrive sequentially through the block-proposer path, so the last contributor sees every prior commitment and can steer the combined key to a chosen point. The repository's design document concedes the bias and argues it is harmless here — the shifted key reveals nothing about the round secret under the discrete-log assumption, and the key serves only ElGamal encryption, whose IND-CPA security holds for any valid key — citing Gennaro et al. (2007) for the security of Joint–Feldman threshold decryption despite a biased key (`vote-sdk/docs/tss-ceremony.md`, "Why single-phase"). The argument is itself designed but unspecified: a repository document by the designers, not a normative artifact or independent analysis.

Remark 3.4 (Threshold collusion)**.** Any t colluding validators can reconstruct the round secret offline and decrypt every individual on-chain ciphertext; summing one vote's sixteen shares then yields that vote's exact per-decision weight, which the cast message already links in public to a VAN nullifier. The linkage stops at the VAN layer — the governance nullifier's keyed PRF keeps Zcash notes and addresses out of reach. No published source claims per-vote weight privacy against a threshold coalition; a specification would have to state this limit explicitly (inference from `crypto/elgamal/bsgs.go` and `proto/svote/v1/tx.proto`, lines 88–99; recorded 2026-07-08).

3.6 Tally: chain-verified threshold decryption

Accumulation. On each accepted reveal, the chain adds the 64-byte ciphertext componentwise into the accumulator keyed by the triple (round, proposal, decision), validating well-formedness and rejecting identity components (`x/vote/keeper/keeper_tally.go`, lines 33–74). By linearity, the accumulator encrypts the summed ballots of all revealed shares for that cell. Nothing decrypts until the round enters its tallying phase.

Partial decryptions. During tallying, each ceremony validator's node injects — through the block-proposer path; the mempool route is rejected — one submission covering every non-empty

accumulator: the partial decryption $D_i := [s_i]\,C_1$ under its DKG share s_i, with a Chaum–Pedersen DLEQ proof that $\log_G \mathsf{VK}_i = \log_{C_1} D_i$. The proof is the pair (e, z); the verifier recomputes $R_1 = [z]\,G - [e]\,\mathsf{VK}_i$ and $R_2 = [z]\,C_1 - [e]\,D_i$ and accepts iff

$$e = H\big(\texttt{svote-pd-dleq-v1} \,\|\, G \,\|\, \mathsf{VK}_i \,\|\, C_1 \,\|\, D_i \,\|\, R_1 \,\|\, R_2\big),$$

with H a BLAKE2b-256 hash-to-scalar; the tag separates this proof family from the tally-level DLEQ family (`svote-dleq-v1`), barring cross-family replay (`crypto/elgamal/dleq.go`, lines 30–145). The verification key VK_i is re-derived on the fly from the round's combined Feldman commitments rather than stored (`x/vote/keeper/msg_server_tally_decrypt.go`, lines 257–269). The DLEQ is what makes the tally trustless against its own operators: one forged D_i would silently shift every total. Duplicate submissions per validator are rejected, and each submission must cover every non-empty accumulator, so a proposer cannot inject a partial set that later wedges the tally (`msg_server_tally_decrypt.go`, lines 199–294).

Final verification. The block proposer alone may submit the claimed plaintext totals. For each entry the chain Lagrange-combines at least t stored partials at zero,

$$\sum_i \lambda_i\, D_i = \Big[\sum_i \lambda_i\, s_i\Big] C_1 = [s]\,C_1,$$

recovering the round secret only in the exponent (`crypto/shamir/partial_decrypt.go`, lines 37–71), and checks $C_2 - [s]\,C_1 = [v]\,G$ against the claimed total v; an empty accumulator forces $v = 0$; any $v \geq 2^{28}$ is rejected outright (the constant `TallyBSGSBound`, an exclusive bound; `x/vote/types/keys.go`, lines 33–35). A completeness check requires the entries to cover every non-empty accumulator — without it a malicious proposer could finalize with results omitted (`msg_server_tally_decrypt.go`, lines 102–177). Only then does the round finalize.

Recovering the logarithm. The chain verifies by re-encoding $[v]\,G$, so only the proposer must solve discrete logarithms, by baby-step/giant-step under $N = 2^{28}$: a table of $m = \lceil\sqrt{N}\rceil = 2^{14} = 16{,}384$ baby steps $j \mapsto [j]\,G$, giant steps subtracting $[m]\,G$, a hit at (i, j) giving $v = im + j$ within at most $m + 1$ iterations (`crypto/elgamal/bsgs.go`, lines 11–110). The bound is ample: the full 21-million-ZEC supply quantizes to 1.68×10^8 ballots at 12,500,000 zatoshi per ballot (`src/params.rs`, line 14), below $2^{28} \approx 2.68 \times 10^8$, so no honest aggregate is ever truncated.

Two flags. Totals are denominated in *ballots*; the protobuf comment on the tally field says zatoshi and is wrong — the wallet multiplies by 12,500,000 for display (`proto/svote/v1/tx.proto`, lines 111–116, against `zashi-android TallyResults.kt`, lines 44–56, commit `b4edd6f6`). And the tallying phase carries a timeout of 21,600 seconds (six hours), after which the round finalizes *without* results (`x/vote/types/keys.go`, lines 28–31) — an availability caveat, not an integrity one, but a specification must decide whether a resultless finalization is acceptable.

What a specification must pin down. Until zips PR #1200 merges, the commit-pinned sources above are the only citable definition of: the Poseidon preimage layouts and the domain-tag registry (currently pinned by unit tests); the three nullifier formulas, their set scoping, and their key encodings; the decrement lookup table and the sentinel-bit convention; the share count, the per-share range bound, and their interaction with partial reveals; the DKG parameter formulas, ceremony state machine, and jailing policy; the DLEQ transcript formats and tags; and the BSGS bound and the unit denomination of totals as consensus parameters.

4 Trust model and verdict

The preceding sections constructed the protocol's mechanisms from commit-pinned source (Table 1); this section assembles what those mechanisms do *not* guarantee. We first map every party the protocol trusts and state exactly what each is trusted with, then classify the volume's claims into the three bins of §1.3, and close with the verdict: what a formal specification would have to pin down before the protocol's guarantees become checkable, and what the protocol delivers today against the instruments that ran the same June 2026 question. File citations below are at the commits of Table 1; wallet files are at `zashi-android` @ `b4edd6f6`.

Every trust relation in this map is itself designed but unspecified: each is concrete, open-source behaviour at the pinned commits, and none is stated in a normative artifact. The inventory of §4.2 records the full classification.

4.1 The trust map

Helper servers hold the vote decision and the linkage. The payload a wallet sends to each chosen helper — struct `SharePayload` in `zcash_voting/src/types.rs` — carries the plaintext `vote_decision`, the vote commitment's `tree_position` in the per-round tree, the `shares_hash`, all sixteen ciphertexts, the sixteen share commitments, and the revealed share's blinding factor (`primary_blind`); the helper builds the Merkle path and the share-reveal proof server-side and submits `MsgRevealShare` itself (`vote-sdk/internal/helper/processor.go`). A helper therefore learns, before any reveal, how the voter voted, and holds precisely the share-to-commitment linkage that the on-chain data blinds; composed with the public pairing of `vote_commitment` and `van_nullifier` in `MsgCastVote` (`proto/svote/v1/tx.proto`), that linkage ties the decision to the vote's delegated authority instance. This is a broader grant than a timing-relay framing suggests. What a helper never receives is weight: the plaintext share values and the ElGamal randomness are marked `SECRET` in the client library and stripped from the wire type (`types.rs`, `EncryptedShare` versus `WireEncryptedShare`). The timing decorrelation itself is client policy, not a protocol guarantee: the wallet posts each share to half of the configured helpers rounded up, each with an independently random future `submit_at` scheduled no later than a last-moment buffer — two fifths of the round duration, capped at six hours — before the vote deadline (`zcash_voting/src/share_policy.rs`); a censoring helper causes partial weight loss, mitigated only by the multi-server fan-out and by a single-share fallback described below.

Any t colluding validators decrypt every posted ciphertext. Function `ThresholdForN` sets the reconstruction threshold $t = \max(2, \lfloor (2n + 2)/3 \rfloor)$ for n ceremony validators (`vote-sdk/x/vote/keeper/keeper_ceremony.go`). Any t of them can reconstruct the round's ElGamal secret key offline from their dealt shares and decrypt every `enc_share` on the chain — not merely the aggregates the tally publishes. The decision attached to each ciphertext is public at reveal time in any case (`MsgRevealShare` carries `vote_decision`); what threshold collusion adds is the share values. In the single-share last-moment mode the exposure is immediate: the client builds a payload for share 0 only, "which carries all the voting weight", and the remaining fifteen zero-value shares are never revealed (`zcash_voting/src/vote_commitment.rs`), so one decryption yields one vote's exact weight. In the sixteen-share mode, recovering a per-vote weight additionally requires grouping shares by vote — a linkage the chain withholds (share nullifiers are blinded), the helpers of the previous paragraph possess outright, and the client-side timing policy merely obfuscates. The linkage stops at the Vote Authority Note layer in every case: the governance nullifier is keyed by the holder's nullifier-deriving key `nk`, so mainnet notes and addresses stay unlinkable even after a later spend (`voting-circuits/src/delegation/circuit.rs`, condition 14). We flag the analytical gap explicitly: neither the public documentation nor the source claims — or denies — per-vote weight privacy against a colluding Election Authority threshold. The limit stated here is our adversarial inference from `crypto/elgamal/bsgs.go`, the

message surface, and the share-range condition of the vote-proof circuit; the property is absent from every published analysis.

The chain is closed and its lifecycle is coordinator-gated. The vote chain enforces a positive-security message whitelist at the ante handler: only enumerated message types pass, standard bank transfers are disabled — explicitly "because unrestricted transfers would let anyone accumulate enough stake to create a validator, bypassing the controlled validator set" (`vote-sdk/proto/svote/v1/tx.proto`, comment on `MsgAuthorizedSend`) — and post-genesis validator creation is disallowed outside the registered-Pallas-key path (`vote-sdk/app/ante_whitelist.go`). Round lifecycle, privileged funding, and software upgrades are threshold-gated *coordinator actions* proposed and approved by a vote-manager set (`MsgProposeCoordinatorAction`, `MsgApproveCoordinatorAction`); the default genesis ships a single vote manager, and the coordinator threshold defaults to one (`vote-sdk/x/vote/module.go`, `DefaultVoteManagerAddresses`; `tx.proto`, `MsgUpdateVoteManagers`). The same coordinators post the snapshot: the eligibility anchors `nc_root` and `nullifier_imt_root` arrive in `MsgCreateVotingSession` as client-fetched values — the note-commitment root recomputed from a lightwalletd frontier, the nullifier root taken from the operator's PIR service (`vote-sdk/api/snapshot.go`) — and the chain stores them without verifying that they describe any real Zcash state; it runs no light client (`x/vote/keeper/msg_server.go`). Honest wallets' proofs fail against fabricated roots, and anyone can audit the posted roots after the fact, but nothing in the protocol prevents a coordinator quorum from pinning a snapshot of its own invention.

The wallet vendor's configuration is the authentication root. A voter's assurance that they are voting in a genuine round reduces to a static configuration file bundled with the wallet: fetched from a commit-pinned URL (`valargroup/token-holder-voting-config @ 2785311d`) and verified against a hard-coded SHA-256 digest, it names the ed25519 keys that must sign the dynamic per-round configuration — helper endpoints, PIR endpoints, and each round's Election Authority key (`StaticVotingConfig.kt`, `BUNDLED_PINNED_SOURCE`). The round authenticator rejects a round whose on-chain `ea_pk` disagrees with the signed entry (`RoundAuthenticator.kt`, status `EA_PK_MISMATCH`) — the wallet's only defence against an attacker-substituted encryption key, under which every cast share would be attacker-decryptable. An on-chain endorsement registry (`MsgEndorseRound`) is secondary. The trust root of the whole client-side protocol is therefore the custodians of the config-signing keys together with the application distribution channel; neither is named in any published artifact.

Tally liveness admits finalization without results. The tally verification itself is real: validators' nodes auto-inject partial decryptions whose Chaum–Pedersen DLEQ proofs are checked on-chain against Feldman-derived verification keys, and the proposer-only `MsgSubmitTally` is accepted only if every claimed total matches the Lagrange combination of stored partials, covers every non-empty accumulator, and lies below the discrete-log recovery bound 2^{28} (`vote-sdk/x/vote/keeper/msg_server_tally_decrypt.go`; `crypto/shamir/feldman.go`; the `TallyEntry` field `decryption_proof` is dead — "reserved for future use" — the binding check is the recomputation). But the threshold t is a liveness parameter as well as a secrecy parameter: if fewer than t validators decrypt, the round sits in `TALLYING` until the timeout — 21,600 seconds by default (`x/vote/types/keys.go`, `DefaultTallyTimeout`) — and the end-blocker then finalizes it with `tally_timed_out = true` and *empty results* (`x/vote/module.go`). A finalized round in which every cast vote remains permanently undisclosed is a protocol outcome, not an excluded failure mode; the design accepts it as the price of not blocking the chain on an unreachable threshold.

The tally wire format mislabels its units. The comment on `TallyEntry.total_value` reads "Decrypted aggregate (zatoshi)" (`vote-sdk/proto/svote/v1/tx.proto:114`). It is wrong: the accumulated plaintexts are *ballot counts* — the vote-proof circuit constrains each vote's shares to sum to the Vote Authority Note's ballot count, and the wallet converts totals to zatoshi by multiplying by the ballot divisor 12,500,000 (`TallyResults.kt`, `toTallyResults`; constant `BALLOT_DIVISOR_ZATOSHI` in `VotingCryptoClient.kt`). The recovery bound corroborates the correct reading: 2^{28} ballots is roughly 33.6 million ZEC — comfortably above the monetary supply — whereas 2^{28} zatoshi is under 2.7 ZEC. A consumer that trusts the comment under-reports every result by a factor of 12.5 million. In a protocol whose only specification is its implementation, a wrong comment in the wire definition is not a cosmetic defect; it is an error in the closest thing the protocol has to a spec.

4.2 Claim inventory: the three bins

Specified. Only the inherited primitives: NoteCommit, nullifier derivation, $\mathsf{Commit}^{\mathsf{ivk}}$, the Sinsemilla Merkle instance, RedPallas, the ZIP-244 sighash, and ZIP-32 derivation, each specified in the protocol specification and the ZIPs, constructed in the *Orchard Guide* (§"Representation of a note: notes and note commitments", §"Prevention of double-spending: nullifiers", §"Keys: capabilities for spending and for viewing", §"Proof of ownership of *some* valid note: the commitment tree", §"Authorisation of the spend: RedPallas"), and reused unchanged through the `unstable-voting-circuits` feature (zcash/orchard @ `c0b6b917`). No governance-specific object reaches this bin.

Designed but unspecified. Everything this volume constructed: the five-message ballot surface; the delegation relation with its note-bundle eligibility and indexed-Merkle-tree non-membership argument; the coordinator-posted, chain-unverified snapshot roots; the Vote Authority Note and its synthetic-action authorization — including the asymmetry that the chain verifies the delegation signature over a *client-provided* sighash, relying on the circuit's ρ-chain for binding, while the cast-vote sighash is recomputed on-chain (`x/vote/ante/validate.go`); all four double-vote layers; the sixteen-share lifted-ElGamal encoding, vote commitment, and bit-mask decrement; the helper protocol and the trust it grants; the per-round Joint–Feldman DKG with threshold $t = \max(2, \lfloor (2n+2)/3 \rfloor)$ and finalization quorum $\lceil 4n/5 \rceil$; the chain-verified tally, its 2^{28} recovery bound, and the timeout path; the PIR eligibility subsystem; and every trust relation of §4.1. Each is concrete and open-source at the pinned commits; none is normative. The would-be specifications — zcash/zips PRs #1200 and #1198, and the prior-art PR #853 — were all open and unmerged as of 2026-07-08.

Open problem. One: the ballot-quantization relation. The delegation circuit constrains $num\_ballots \cdot 12{,}500{,}000 + remainder = v_{\text{total}}$ with $remainder < 2^{24}$, which is strictly weaker than the floor division the public documentation states: the pair $(q-1,\ r+12{,}500{,}000)$ also satisfies every constraint whenever the honest remainder obeys $r < 2^{24} - 12{,}500{,}000 = 4{,}277{,}216$ — about 34.2% of residues — and the honest ballot count is at least two, while the opposite deviation fails the range check, so a prover can under-claim exactly one ballot and can never over-claim (`voting-circuits/src/delegation/circuit.rs`; `src/delegation/README.md` §8, which states the window, proves over-claim impossible, and lists unimplemented tightening constraints). The deviation is self-harming, but it is the volume's one confirmed disagreement between the code and the code's own public description, and the exact relation a specification must choose — floor division or the shipped weaker relation — is unresolved.

4.3 Verdict

What a specification must pin down. For the protocol's guarantees to become checkable — by an auditor, by a second implementation, or by this volume — a normative artifact would need to fix, at minimum:

1. the three circuit relations in full: public-input layouts, every condition, and the exact quantization rule — either closing the under-claim window or normatively accepting the shipped relation;
2. every governance-object formula: the four nullifiers and the nullifier-domain derivation, the Vote Authority Note commitment (with the argument that its x-only encoding is bound elsewhere), the vote commitment, and the shares hash;
3. the snapshot: how `nc_root` and the nullifier root are derived, and *who verifies them* — at present, on-chain, nobody;
4. the authorization model: which sighashes the chain recomputes, which it accepts from the client, and why the delegation path is safe;
5. the Election Authority: the threshold and quorum formulas, key registration and rotation, and — the largest analytical gap — an explicit confidentiality claim stating against whom per-vote weight privacy holds;
6. the helper interface: the exact payload, the trust granted by it, and the status of the timing policy as guarantee or convention;
7. tally semantics: the verification equation, the unit of `total_value` (ballots), and the meaning of a timed-out round;
8. independent verification: a second implementation of at least the verifiers, breaking the single-crate tautology of §1.3.

What the protocol delivers today. Measured against the instruments beside it (§1.2), the deliverable is real but conditional. Against ZCAP, the protocol opens participation to every Orchard coinholder in proportion to holdings, with no identified panel and no registrar, and it hides identity and weight from public observers — where ZCAP offers the converse trade: an identified panel on mature, independently analysed voting software. Against the prior coin-weighted instrument, hhanh00's `zcash-vote`, it adds threshold decryption in place of a single tallying authority, blinded delegation through the Vote Authority Note, hardware signing, PIR-private eligibility queries, and wallet-native deployment — at the price of the trust map of §4.1: a permissioned chain, a coordinator quorum, helper servers, and a vendor-held configuration root. Integrity of the aggregate totals is the strongest property: the tally check is executed on-chain and every input to it is public, so any observer of the vote chain can replay it, and a fabricated snapshot is auditable after the fact. Individual privacy is the weakest: it holds against outside observers, degrades to helper honesty for the linkage of decisions to votes, and degrades to the honesty of all but $t - 1$ validators for individual weights — a limit no published analysis states. On that evidence the protocol is fit for what it was used for in June 2026: a non-binding sentiment poll with publicly recomputable totals. Binding governance would additionally require the specification above, an independent verifier, and a confidentiality analysis of the Election Authority — none of which existed as of 2026-07-08.